PARK IT: DATABASE MANAGEMENT SYSTEM

PROJECT DEVELOPMENT DOCUMENTATION

Group Number: 2
Course / Section: BSIT 3-1
Instructor / Adviser: John Kenneth Timblaco
Documentation Date: October 25, 2026

Group Members and Roles

Name	Assigned Role
De Vera Miller P.	Project Manager
Butanas John David	UI/UX
Macascas Mark Andrew	Front-end
Pelia Jhon Francis	Back-end
Lumbo Marianne	Quality Assurance

Project Repository:
https://github.com/HelloP0/PARK-IT-DATABASE-MANAGEMENT-SYSTEM.git
Project Tracker: [BSIT 3-1] - PROJECT TRACKER.xlsx - Google Sheets

---

1. SYSTEM OVERVIEW

1.1 Project Introduction

Parking management can be challenging, especially when monitoring vehicle entries and exits, checking available parking spaces, and handling parking payments manually. These processes can be time-consuming and may lead to errors in recording information and managing parking operations.

To address these challenges, the developers proposed **Park It: Database Management System**, an automated system designed to improve and simplify parking management. The system provides an organized way of recording vehicle entries and exits, calculating parking fees, monitoring real-time parking space availability, and verifying payment statuses for each vehicle.

Through automation and database management, the system aims to make parking operations more convenient, accurate, and efficient.

1.2 Purpose of the Project

The purpose of Park It is to develop an automated database system that improves the management and monitoring of parking operations. It aims to reduce manual work, minimize recording errors, and provide accurate parking information.

Specifically, the system is designed to:

1. Automatically record vehicle entry and exit information.
2. Calculate parking fees based on the vehicle's parking duration.
3. Monitor and display available and occupied parking spaces in real time.
4. Securely verify and record payment statuses for each vehicle.
5. Provide an organized database for storing and managing parking records.

1.3 Project Scope

The scope of Park It focuses on parking management and database operations. It covers vehicle record management, parking entry and exit monitoring, parking fee calculations, parking space availability monitoring, and payment status verification.

The system also includes database operations that allow authorized users to create, retrieve, update, and delete records.

The project is intended to provide an organized and efficient parking management solution for parking administrators and vehicle owners.

1.4 Main System Functions

Function	Description
Vehicle Entry	Records vehicle information and entry time.
Vehicle Exit	Records vehicle exit time.
Parking Fee Calculation	Calculates fees based on parking duration.
Parking Space Monitoring	Displays available and occupied spaces.
Payment Verification	Records and verifies payment status.
Record Management	Allows authorized users to add, view, update, and delete records.

---

2. USER ROLES

This section identifies the intended users of the application, their responsibilities, and their access to the system's data.

2.1 Application User Roles

User Role	Responsibilities	Data Access

Administrator	Manages application users, parking records, and system operations.	Manage parking records and user accounts, subject to implemented permissions.
Parking Attendant	Handles vehicle entries, exits, and parking transactions.	Access and manage vehicle and parking transaction records required for daily operations.
Vehicle Owner	Uses the parking service and receives parking and payment information.	View relevant parking and payment information.

---

3. DEVELOPMENT PROGRESS LOG

This section documents the development activities performed by the group, including completed tasks, ongoing activities, responsible members, and relevant repository commits.

3.1 Development Progress

Development Activity	Status	Progress
Project Planning	Completed	100%
Requirements Gathering	Completed	100%
Database Design	Completed	100%
ERD Development	Completed	100%
Database Creation	Completed	100%
User Interface Design	Ongoing	50%
Database Connection	Completed	80%
CRUD Implementation	Ongoing	60%
System Testing	Ongoing	50%
Documentation	Ongoing	30%

3.2 Development Log

Development Activity	Responsible Member/s	Status
Initial project planning and requirements gathering	**De Vera Miller P.**	Completed
Database design and ERD development	**De Vera Miller P.**	Completed
Database creation and configuration	**De Vera Miller P.**	Completed

Database connection implementation	**De Vera Miller P., Pelia Jhon Francis**	Completed
CRUD functionality development	**Macascas Mark Andrew, Pelia Jhon Francis**	Ongoing
User interface improvement	**Butanas John David, Macascas Mark Andrew**	Ongoing
System testing and debugging	**Lumbo Marianne**	Ongoing

3.3 Repository and Commit History

The group uses Git and a shared repository to manage the project's source code and track development changes.

Key Commits

- Initial project files
- Database connection implementation
- CRUD functionality updates
- Interface improvements

---

4. DATABASE IMPLEMENTATION

4.1 Database Design

The database design phase focuses on identifying the necessary tables, fields, relationships, and constraints required by the Park It application.

The database is designed to organize vehicle information, parking records, parking spaces, and payment transactions.

Database Name: grp_parkitdb
Number of Tables: 5
Primary Keys Established: Yes
Foreign Keys Established: Yes
Relationships Implemented: Yes

4.1.1 Entity Relationship Diagram (ERD)

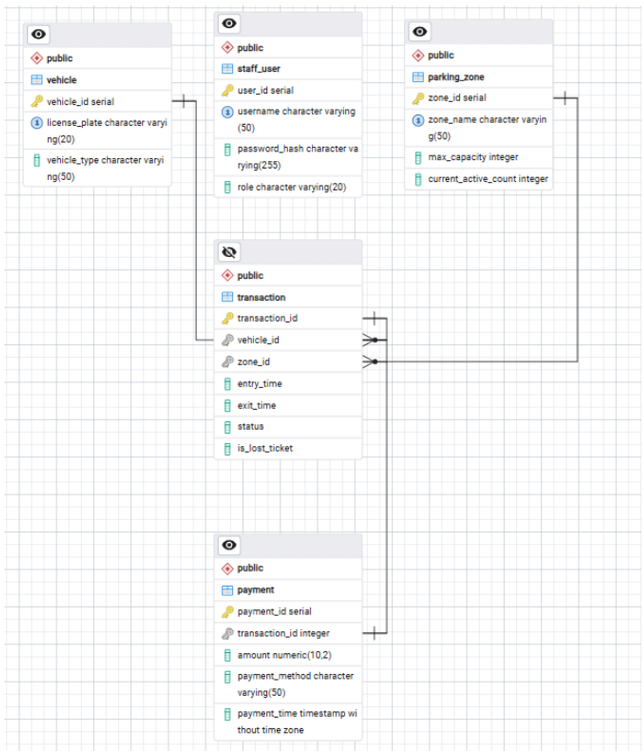


Figure 1. Entity Relationship Diagram of the Park It Database.

4.2 Database Schema Summary

The database contains tables responsible for storing and managing the information required by the parking application.

Table Name	Description	Primary Key
parking_zone	Stores parking zone information, including zone details and capacity.	zone_id
vehicle	Stores vehicle information, including license plate details.	vehicle_id
transaction	Records vehicle parking transactions, including entry and exit information and transaction status.	transaction_id
payment	Stores payment information associated with parking transactions.	payment_id
staff_user	Stores staff account information used by the system.	user_id

4.3 SQL Scripts and Migrations

The database implementation uses SQL scripts or migration files to create and configure the necessary database structures.

The scripts are responsible for:

- Creating the database.

- Creating the required tables and columns.
- Establishing primary and foreign keys.
- Defining relationships between tables.
- Applying constraints to maintain data integrity.
- Inserting initial or sample records.

SQL Script / Migration Files:

```
CREATE TABLE public.parking_zone (
    zone_id integer NOT NULL,
    zone_name character varying(50) NOT NULL,
    max_capacity integer NOT NULL,
    current_active_count integer DEFAULT 0 NOT NULL,
    CONSTRAINT parking_zone_check CHECK ((current_active_count <= max_capacity))
);

CREATE TABLE public.payment (
    payment_id integer NOT NULL,
    transaction_id integer NOT NULL,
    amount numeric(10,2) NOT NULL,
    payment_method character varying(50) NOT NULL,
    payment_time timestamp without time zone NOT NULL,
    CONSTRAINT payment_amount_check CHECK ((amount >= (0)::numeric))
);


CREATE TABLE public.staff_user (
    user_id integer NOT NULL,
    username character varying(50) NOT NULL,
    password_hash character varying(255) NOT NULL,
    role character varying(20) NOT NULL
);


CREATE TABLE public.transaction (
    transaction_id integer NOT NULL,
    vehicle_id integer NOT NULL,
    zone_id integer NOT NULL,
    entry_time timestamp without time zone NOT NULL,
    exit_time timestamp without time zone,
    status character varying(20) DEFAULT 'Active'::character varying NOT NULL,
    is_lost_ticket boolean DEFAULT false NOT NULL
);

CREATE TABLE public.vehicle (
    vehicle_id integer NOT NULL,
    license_plate character varying(20) NOT NULL,
    vehicle_type character varying(50) NOT NULL
);
```

4.4 Seed Data

Seed Data	Purpose	Status
Sample vehicle records	Testing vehicle information storage	Ongoing
Sample parking records	Testing parking transactions	Ongoing
Sample parking spaces	Testing space availability	Ongoing
Sample payment records	Testing payment statuses	Ongoing

---

5. APPLICATION EVIDENCE

This section provides screenshots and testing evidence demonstrating the actual implementation and functionality of the Park It application.

5.1 User Interface Evidence

Login Interface



Figure 3. Login interface of the Park It application.

Main Dashboard

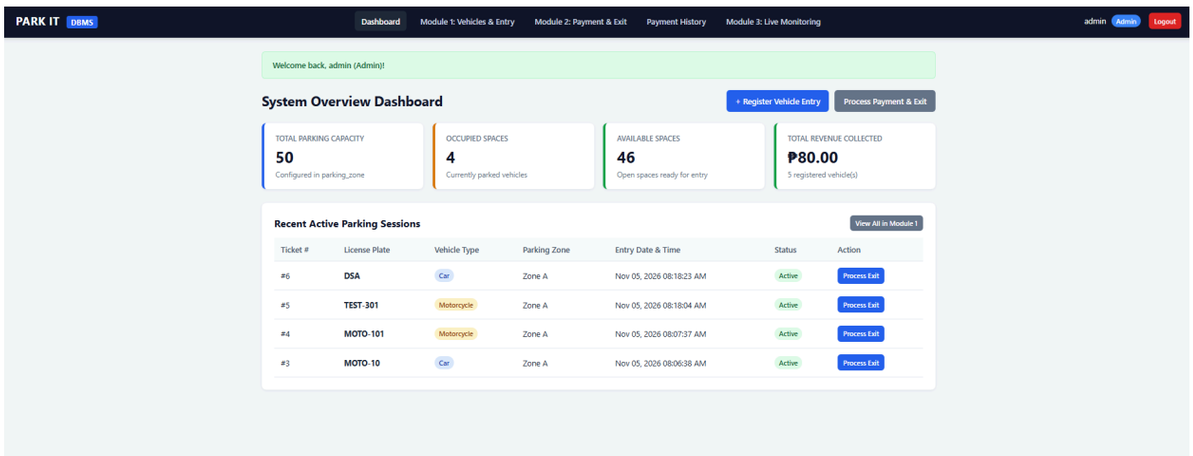


Figure 4. Main dashboard displaying parking information.

Vehicle Record Management

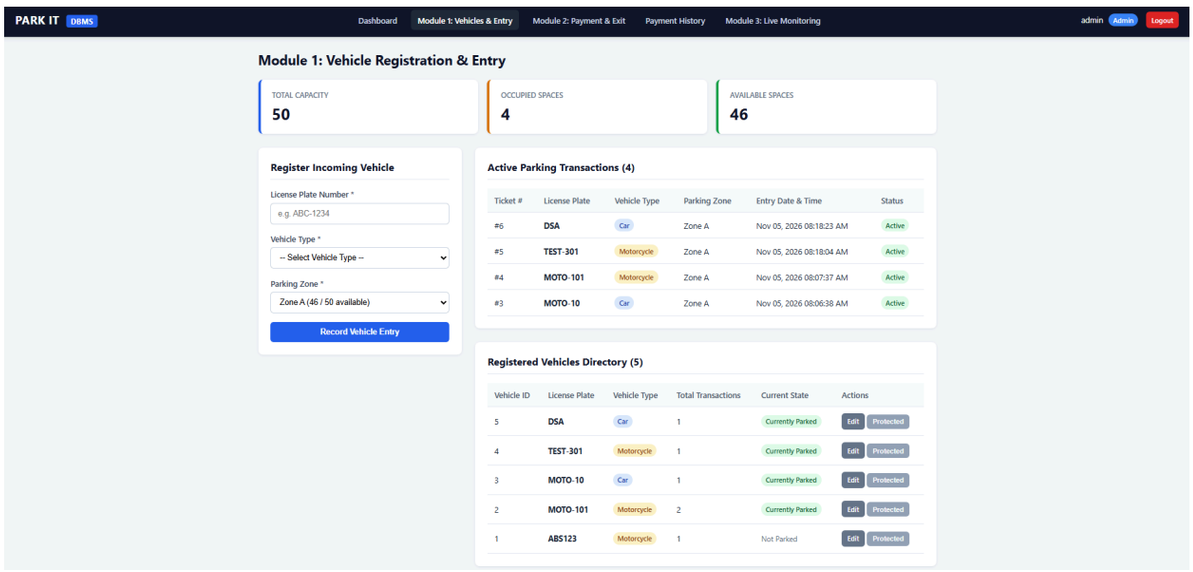


Figure 5. Vehicle record management interface.

Parking Space Monitoring

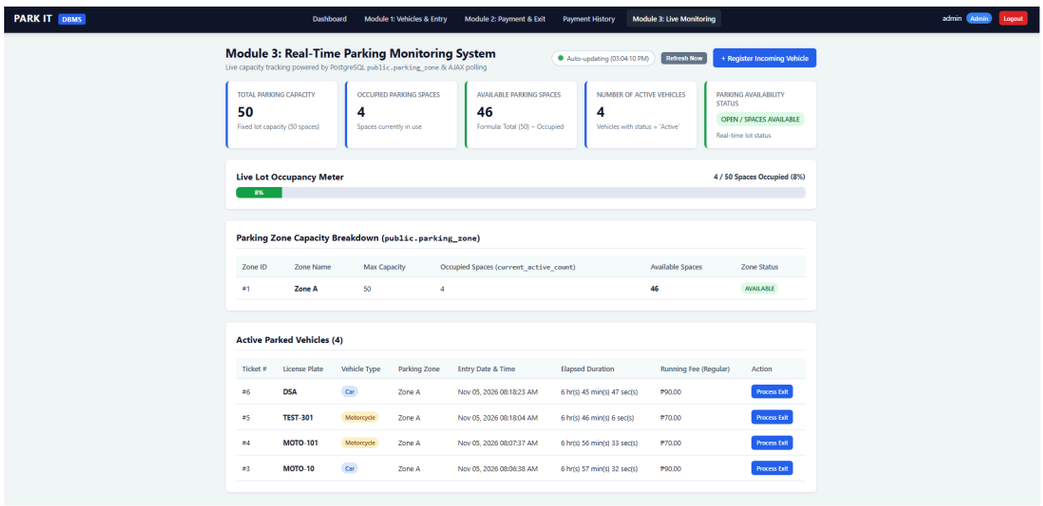


Figure 6. Parking space availability interface.

Payment Management

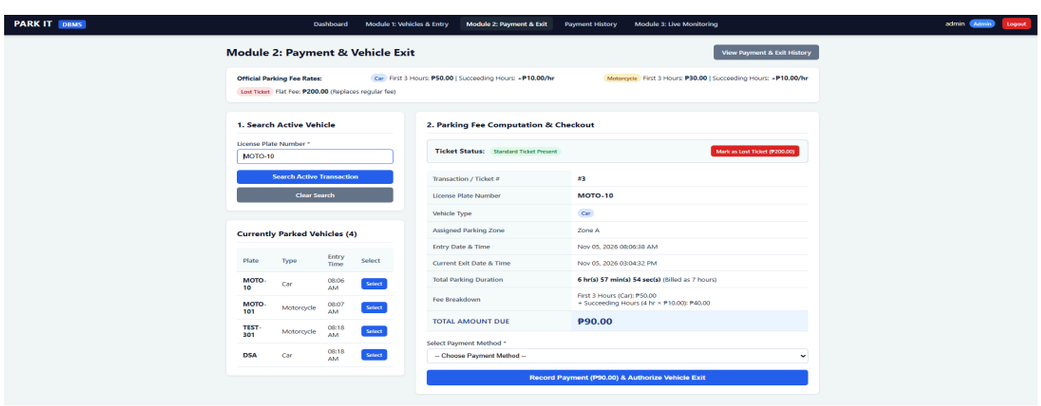


Figure 7. Payment information and verification interface.

5.2 CRUD Functionality Testing

The following tests are conducted to verify that the application can properly manage database records.

Test	Expected Result	Status
Create	New records are saved in the database.	Completed
Retrieve	Existing records are displayed.	Completed
Update	Existing records are modified successfully.	Completed
Delete	Selected records are removed successfully.	Completed

5.2.1 Create Test

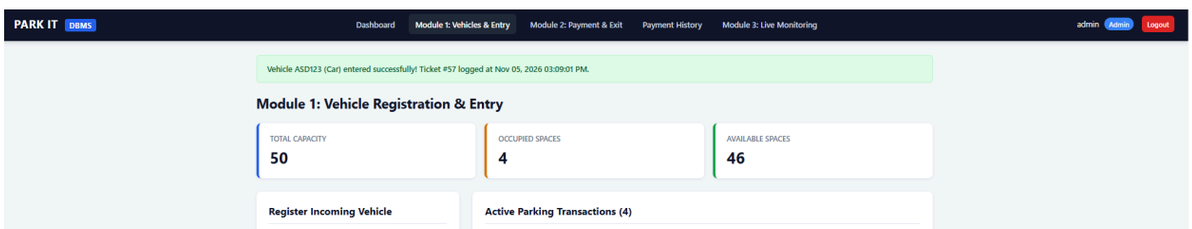


Figure 8. Successful creation of a database record.

5.2.2 Retrieve Test

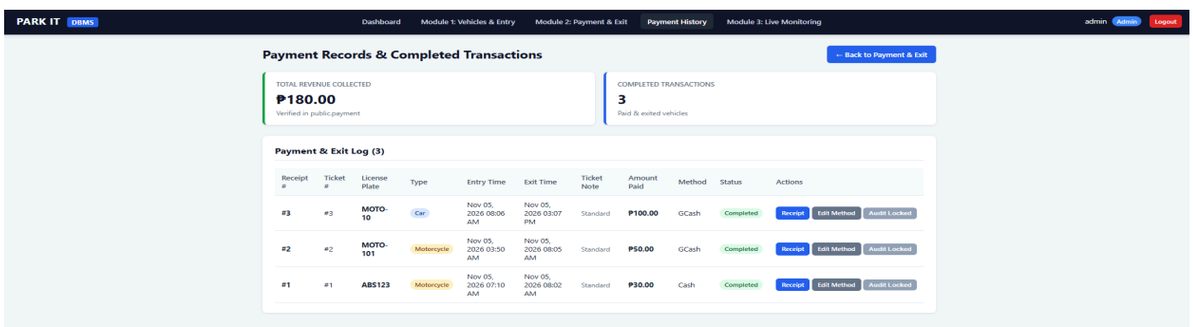


Figure 9. Retrieved records displayed in the application.

5.2.3 Update Test

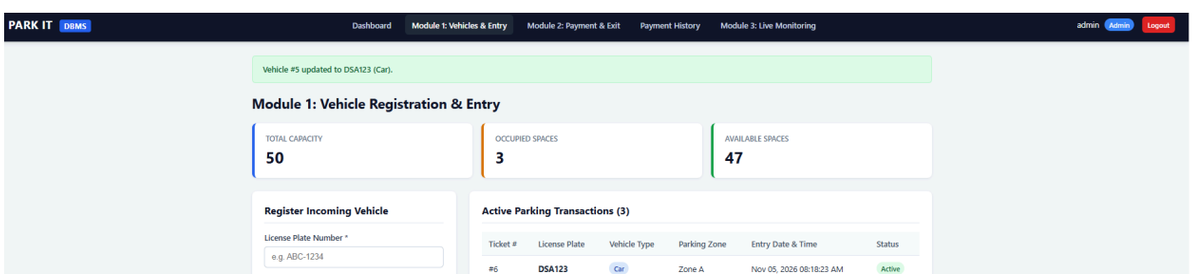


Figure 10. Successful updating of an existing record.

5.2.4 Delete Test

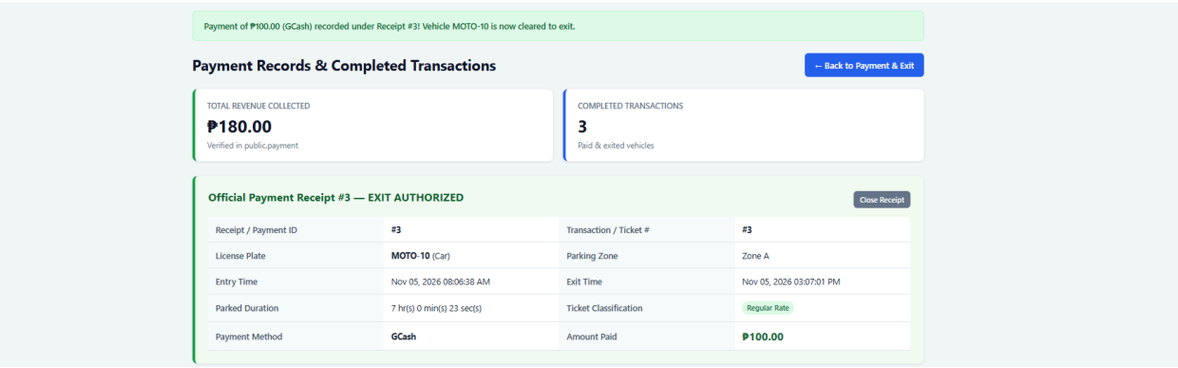


Figure 11. Successful deletion of a selected record.

---

6. SETUP AND CONFIGURATION

This section provides the basic instructions required to install, configure, and run the Park It application.

6.1 Prerequisites

The following tools and software are required to run the application:

- Node.js
- Git
- PostgreSQL
- Preferred IDE

6.2 Installation Steps

1. Install the required software and development tools.
2. Clone the Park It repository from GitHub.
3. Open the project folder in the development environment.
4. Install the required project dependencies.
5. Configure the database connection.
6. Create the database using the provided SQL scripts or migration files.
7. Insert the required seed data.
8. Start the application using the project's configured development command.
9. Open the application in your preferred browser.

Repository: https://github.com/HelloP0/PARK-IT-DATABASE-MANAGEMENT-SYSTEM.git

6.3 Environment Variables

The application may require environment variables to configure its database connection and application settings.

Example placeholders:

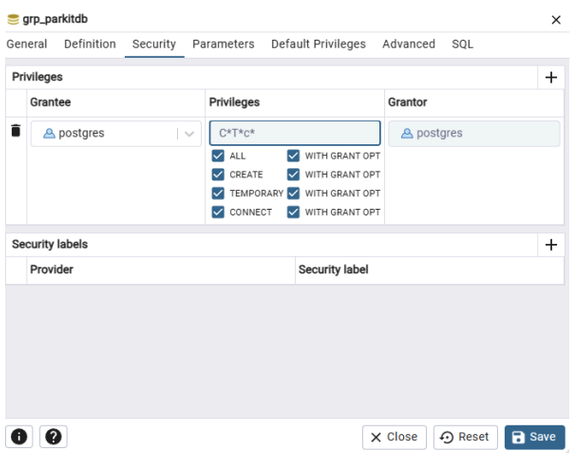
Variable	Description	Example Placeholder
DB_HOST	Database server address	[DB_HOST]
DB_USER	Database username	[DB_USER]
DB_PASSWORD	Database password	[DB_PASSWORD]
DB_NAME	Database name	[DB_NAME]
DB_PORT	Database port	[DB_PORT]
PORT	Application port	[PORT]

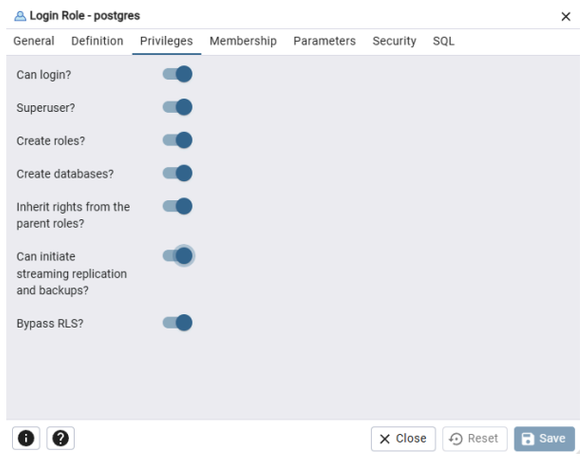
6.4 Database User and Role Setup

Database accounts must be configured to allow the application to connect to the database securely.

The database setup should include:

- Creating the necessary database accounts.
- Assigning appropriate permissions to each account.
- Configuring the application's database connection.
- Testing whether the application can successfully access the database.





---

7. ISSUES AND RESOLUTIONS

During development and setup, the group encountered several technical challenges. The following table documents the problems and the solutions applied.

Problems Encountered	Solutions Applied
The initial interface was static.	Implemented CRUD functionality to make the system more interactive and allow database record management.
Node.js and Git were not recognized in the terminal due to path variable issues.	Checked and corrected the installation paths, then restarted the development environment.
Git merge conflicts during the final push.	Resolved the branch conflict by updating the main branch using the working local copy.

7.1 Current Limitations

The following are the identified areas that still require improvement:

1. **Basic User Interface** – The current interface is still too plain and needs further improvement in terms of design, layout, colors, and overall appearance to make it more attractive and user-friendly.
2. **Incomplete Functionalities** – Some features are still under development and may not yet function as intended.
3. **Input Validation** – Some input fields may still need additional validation to prevent incorrect, incomplete, or duplicate vehicle records.
4. **System Testing** – The application has not yet undergone complete testing, so some errors or unexpected issues may still occur.
5. **Mobile Responsiveness** – The current interface may not yet be fully optimized for different screen sizes, particularly mobile devices.

These limitations will be addressed during the succeeding development phases.

---

8. AI USAGE DECLARATION

This section declares the use of artificial intelligence tools during the preparation and development of the Park It project.

8.1 AI Tools Used

AI Tool	Purpose of Use
ChatGPT	Assistance in preparing, organizing, and improving project documentation.
Gemini, Claude	Assistance in developing the database and in making the overall system.

8.2 Tasks Assisted by AI

AI assistance was used for documentation-related activities, including:

- Organizing the project documentation into the required sections.
- Improving the grammar, clarity, and structure of written content.
- Providing suggestions for project overviews, objectives, limitations, and conclusions.
- Suggesting ways to present technical information clearly.
- Generating the system.
- Assisting in creating the database.

8.3 Validation and Verification

All AI-assisted content is reviewed by the group before being included in the project documentation. The team is responsible for checking the accuracy of the information and ensuring that the written content reflects the actual implementation and development progress of Park It. AI-generated suggestions are not treated as automatically correct. Any inaccurate or unsuitable information is corrected or removed based on the actual project requirements and system implementation. The group remains responsible for the final project, its source code, database implementation, and submitted documentation.
